ci1316 - PROGRAMAÇÃO PARALELA -
2o semestre de 2026
por W.Zola/UFPR

Trab 2: Multiplicaçao de matrizes com MPI

Histórico:
- v1.0 a versão inicial

Data do enunciado: 5/nov/2026
Data da entrega: **20/nov/2026**

Objetivo do trabalho:
O objetivo deste exercício é fazer um trabalho que use paralelismo em MPI no cluster com rede ethernet e verificar a aceleração da nossa solução.

.O trabalho:

O trabalho consiste de duas versões paralelas (A e B) para a multiplicação paralela de matrizes com MPI.

- A versão A é uma versão paralela normal (simples, direta) da multiplicação de matrizes com MPI.
- A versão B (B = **B**locada) é uma versão paralela MPI que tenta aproveitar o uso de cache para melhorar o desempenho do algoritmo. Ver slides com ideias da versao blocada.
- Haverá APENAS um programa fonte em na linguagem C, que roda a versão A ou a versão B, dependendo dos parâmetros na linha de comando.

A implementação DEVE seguir as idéias algoritmo descritas aqui. Para simplificar, o paralelismo deve usar MPI apenas. As operações coletivas como, por exemplo, MPI_Bcast e MPI_Scatter (e/ou MPI_Gather) devem ser usadas para distribuir as matrizes entre os nodos de processamento, e para obter, no nodo 0, a matriz resultante da multiplicação de duas outras.

Sobre o uso de primitivas MPI

Voce DEVE usar as melhores primitivas de MPI adequadas ao trabalho. O envio bloqueante em uma etapa de transmissao apenas, tem, possivelmente um pior desempenho. Possivelmente na versao B, ao tentar usar operacoes não bloqueantes e envio em blocos seria, teoricamente, melhor para se obter desempenho, tentando sobrepor computacao e comunicacoes.

Multiplicação de matrizes com paralelismo em MPI – idéias iniciais

Dadas duas matrizes quadradas A e B, de ordem [Nla x M] e [M x Ncb] respectivamente. Queremos obter a matriz C de ordem [Nla x Ncb], resultante da multiplicação de A * B.

As matrizes A, B e C devem ser de números DOUBLE.

Para simplificar, e evitar leitura de disco, faremos A inicialização das matrizes A e B no nodo 0.

A partir de parâmetros na da linha de comando o programa deve rodar os processos MPI que farão a execução do multiplicação paralela.

Se for informada a opção -v na linha de comando, o programa deve rodar uma versão sequencial do algoritmo e comparar o resultado com a versao paralela (comparar SE multiplicou corretamente).

Vamos colocar o nome “mmul.c” no nosso programa. E “mmul” será o programa executável.

MODOS DE EXECUÇAO

O programa podera ser rodado no modo de MEDICAO (i.e. modo NORMAL de execuçao e mediçao de desempenho, SEM verificacao) ou no modo VERIFICADOR (que apenas inclui uma etapa de verificacao de correcao do algoritmo). O modo verificado deve ser ativado SE o parametro -v for dado na linha de comando. A separacao em dois modos eh apenas para que se possa fazer a verificacao em exeucao separada (sem necessidade de slurm para verificacao, apenas MPI). Desta forma, a verificacao de correcao eh feita no nodo w00 do cluster de maneira separada, SEM usar slurm, pois a inclusao de verificacao poderia ultrapassar o tempo da fila slurm.

Para rodar a versao A (simples) no modo de MEDICAO, fazer (rodar e medir APENAS, usando slurm):
IMPORTANTE:
Veja nas informacoes na UFPR Virtual como rodar slurm corretamente com MPI.
Abaixo coloque apenas mpirun, mas voce DEVE adaptar para rodar com script slurm no cluster.

mpirun -np n Nla M Ncb

Isso roda o a versão paralela com n processos MPI, inicializando as
duas matrizes quadradas A e B, de ordem [Nla x M] e [M x Ncb]
Respectivamente e fazendo a multiplicação paralela.
Nessa opção, o programa deve imprimir:
a) CADA processo MPI deve imprimir:
o seu rank e o nome do host que rodou.
b) o tempo gasto na multiplicaçao paralela apenas (reportar em segundos) i,e, tempo total, e os tempos de comunicacao e computacao separados.
c) a VAZAO obtida no cálculo (em GFLOPS)
Note que GFLOPS = Giga Floating Point Operations per Second
(ou seja, o S de GIGAFLOPS significa “por segundo”)

OBS: nesse modo, se rodar com -np 1,
o MPI deve rodar apenas no nodo 0, a versão SEQUENCIAL e reportar tempos, e GFLOPS sequenciais.

Para rodar a versao B (blocada) no modo de MEDICAO, fazer (rodar e medir APENAS, usando slurm):
IMPORTANTE:
Veja nas informacoes na UFPR Virtual como rodar slurm corretamente com MPI.
Abaixo coloque apenas mpirun, mas voce DEVE adaptar para rodar com script slurm no cluster.

mpirun -np n Nla M Ncb **-w blockWidth**

Isso roda o a versão paralela blocada com blocos QUADRADOS de lagura **blockWidth elementos**, com n processos MPI, inicializando as duas matrizes quadradas A e B, de ordem [Nla x M] e [M x Ncb] Respectivamente e fazendo a multiplicação paralela.
BlockWidth deve ser informado largura do bloco QUADRADO de elementos DOUBLE que cabem no bloco e no cache
OBS:

Possivelmente, o melhor valor de w (blockWidth) a ser usado nas experiencias deve ser obtido de maneira a conseguir posicionar DENTRO do cache: tres blocos: um da matriz A, outro de B e outro da matriz C.
ASSIM, ao executar em blocos, cada computacao traz os blocos de A, B e calcula um bloco de C, tudo estando em cache e melhorando o desempenho.

Nessa opção, o programa deve imprimir:
a) CADA processo MPI deve imprimir:
o seu rank e o nome do host que rodou.
b) o tempo gasto na multiplicaçao paralela apenas (reportar em segundos) i,e, tempo total, e os tempos de comunicacao e computacao separados.
c) a VAZAO obtida no cálculo (em GFLOPS)
Note que GFLOPS = Giga Floating Point Operations per Second
(ou seja, o S de GIGAFLOPS significa “por segundo”)

OBS: nesse modo, se rodar com -np 1,
o MPI deve rodar apenas no nodo 0, a versão SEQUENCIAL e reportar tempos, e GFLOPS sequenciais.

Simplificacao da versao B

Voce pode fazer uma versao B (blocada) GENERICA sem restricoes de tamanhos e valores de w.

OU, caso queira fazer uma versao mais facil
voce pode colocar as seguintes restricoes no seu programa, no inicio,
depois de ler a linha de comando:

Obtenha os valores
Nla // numero de linhas de A (e de C)
M // numero de colunas de A = linhas de B
Ncb // numero de colunas de B (e de C)

Exija que esses valores seja restritos a:

```
// Restrições do exercício: colunas múltiplas de 8 (linhas alinhadas)
// e Nla divisível por nprocs.
if (M % 8 || Ncb % 8 || Nla % nprocs) {
    if (rank == 0) fprintf(stderr, "erro: M e Ncb devem ser multiplos de 8 "
                                   "e Nla divisivel por nprocs\n");
    MPI_Finalize();
    return 1;
}
```

Para rodar, fazer (MODO VERIFICADOR, nas versoes A ou B, rodar e verificar correção do resultado APENAS, sem medir tempo.)

versao A:
mpirun -np n Nla M Ncb -v
OU

versao B:
mpirun -np n Nla M Ncb -w blockWidth

Isso roda o a versão paralela com n processos MPI, inicializando as
duas matrizes quadradas A e B, de ordem [Nla x M] e [M x Ncb]
Respectivamente e fazendo a multiplicação paralela.

Ao final, no nodo 0, roda uma versão SEQUENCIAL do algoritmo e
compara a correção do resultado paralelo em relação ao serial.

Nessa opção -v, o programa deve imprimir:
a) Se o resultado paralelo foi considerado correto.
b) alguma medida de erro (se houver)

Mediçoes:

Vamos medir com processos EXCLUSIVOS, no cluster Xeon, na fila de processos (comando sbatch). Ou seja, CADA processo MPI deve rodar em um nodo separado e rodar sozinho no nodo (modo EXCLUSIVO)

As medicoes de tempo DEVEM ser feitas com a API de cronometro (chono.h ou chrono.c) dada pelo prof. no trabalho ANTERIOR ou exemplos de reducao paralela.

O programa DEVE informar separadamente o tempo gasto em COMPUTACOES e o tempo gasto em COMUNICACOES, e o tempo total. Provavelmente, seria mais simples e adequado medir separadamente apenas o tempo total e o tempo de computacoes, e REPORTAR o tempo de comunicacao com a diferença entre os dois tempos. O tempo total da multiplicacao deve ser usado para reportar a vazao em GFLOPS, ou seja, a vazao de operacoes de pronto flutuante (que é uma medida de vazao que faz sentido para a multiplicacao de matrizes).

Medir:
- Em principio, vamos medir para matrizes de doubles

$$\mathbf{A = 2304 \times 960,\ B = 960 \times 960,\ C = 2304 \times 960}$$

Se esses tamanhos demorarem muito ou forem muito rápido, vamos mudar, mas acho que esses tamanhos serão adequados..
Voce deve obter o melhor valor para w (blockWidth) e fazer experiencias com esse valor.
DICA: o prof acha que o melhor w é 192

-Vamos medir 10 x cada experimento e reportar a média das medidas
-Vamos medir para 1 nodo (sequencial), 2 nodos, 3 nodos e 4 nodos.
- Fazer a planilha como nos trabalhos anteriores, incluindo todas as infos, medidas, médias, graficos ADEQUADOS e tabelas de tempo e aceleraçao.
- fazer e entregar relatório nos moldes dos trabalhos anteriores,
com conclusões.

- Entrega:
Entrega será na UFPR virtual em método similar aos trabalhos anteriores.

Nas próximas paginas temos figuras com ideias sobre os algoritmos.

SOBRE AS EXPERIÊNCIAS:

- Faremos a computacao para:

- **Experiencia A:**
- Rodar o programa da versao A APENAS e mostrar tabela com valores de vazao e aceleracoes obtidas em relacao a 1 processo MPI para 1, 2, 3 e 4 processos MPI (rodando simultanemente em nodos distintos)

- **Experiencia B:**
- Rodar o programa da versao B APENAS e mostrar tabela com valores de vazao e aceleracoes obtidas em relacao a 1 processo MPI para 1, 2, 3 e 4 processos MPI (rodando simultanemente em nodos distintos)

- **Comparações de A com B:**
- mostre na sua planilha uma folha extra com uma tabela com comparacoes de A e B, e inclua a aceleracao de B em relacao a A, para 1, 2, 3 e 4 processos MPI (rodando simultanemente em nodos distintos)
- inclua e DISCUTA essa comparacao de versoes paralelas no seu relatório TAMBEM

- **SOBRE O RELATÓRIO do trabalho:**

  - Fazer um relatório (PDF) com suas conclusões. Descreva as características importantes da sua CPU usada. Inclua em apendice ao relatório o texto da saida do comando lscpu bem como a figura do comando lstopo para sua CPU usada.

- **SOBRE A funçao de vefificaçao a ser chamada**

  - Incluir ao final do seu programa, ANTES de terminar o MPI e ANTES de desalocar suas matrizes, para rodar APENAS no processo 0 MPI, que DEVE ter o mesmo prototipo abaixo.

```
verifica_resultado -- confere C = A x B contra uma multiplicação sequencial
A  [Nla x M]     matriz A completa
B  [M x Ncb]     matriz B completa
C  [Nla x Ncb]   resultado a conferir (já reunido pelo MPI_Gatherv)
Todas as matrizes são de double, armazenadas por linhas (row-major).
Retorna o número de elementos de C diferentes da referência:
0 = resultado correto.
Se houver erro, imprime em stderr o primeiro elemento errado e o total.
Regras de uso:
    • chamar só no rank 0, FORA da medição de tempo;
    • a comparação é EXATA (==), o que só vale porque as entradas são
inteiros pequenos guardados em double (ver gera_matriz);
    • aloca internamente uma matriz auxiliar de Nla x Ncb doubles.
```

PROTOTIPO:

- size_t verifica_resultado(int Nla, int M, int Ncb, const double *A, const double *B, const double *C);

GERACAO DAS MATRIZES

Vamos gerar matrizes conforme abaixo. PORQUE ? Porque com esses valores pequenos vai facilitar a verificacao dos resultados (sem erros)

```
/*
 * gera_matriz -- preenche uma matriz com inteiros pequenos em [-4, 4]

 *   X  vetor de n doubles (a matriz inteira, armazenada por linhas)
 *   n  número de elementos: linhas x colunas
 *
 * Por que inteiros pequenos guardados em double:
 *   cada produto vale no máximo 16 em módulo, e cada elemento de C no máximo
 *   16*M. Esses valores, e todas as somas parciais, são inteiros representados
 *   EXATAMENTE em double (bem abaixo de 2^53). Assim o resultado não depende
 *   da ordem das somas, e a verificação pode comparar com ==, sem tolerância,
 *   mesmo que a versão blocada some os termos numa ordem
 *          diferente da sequencial.
 *
 * Regras de uso:
 *   - chamar só no rank 0, antes de enviar para os nodos;
 *   - fixar a semente com srand(1) ANTES da primeira chamada, para que toda
 *     execução use as mesmas matrizes e um erro possa ser reproduzido.
 */
static void gera_matriz(double *X, size_t n)
{
    for (size_t i = 0; i < n; i++)
       X[i] = (double)(rand() % 9 – 4);
             /* rand() % 9 dá 0..8; menos 4 dá -4..4 */
}
```

No `main`, depois da alocação:

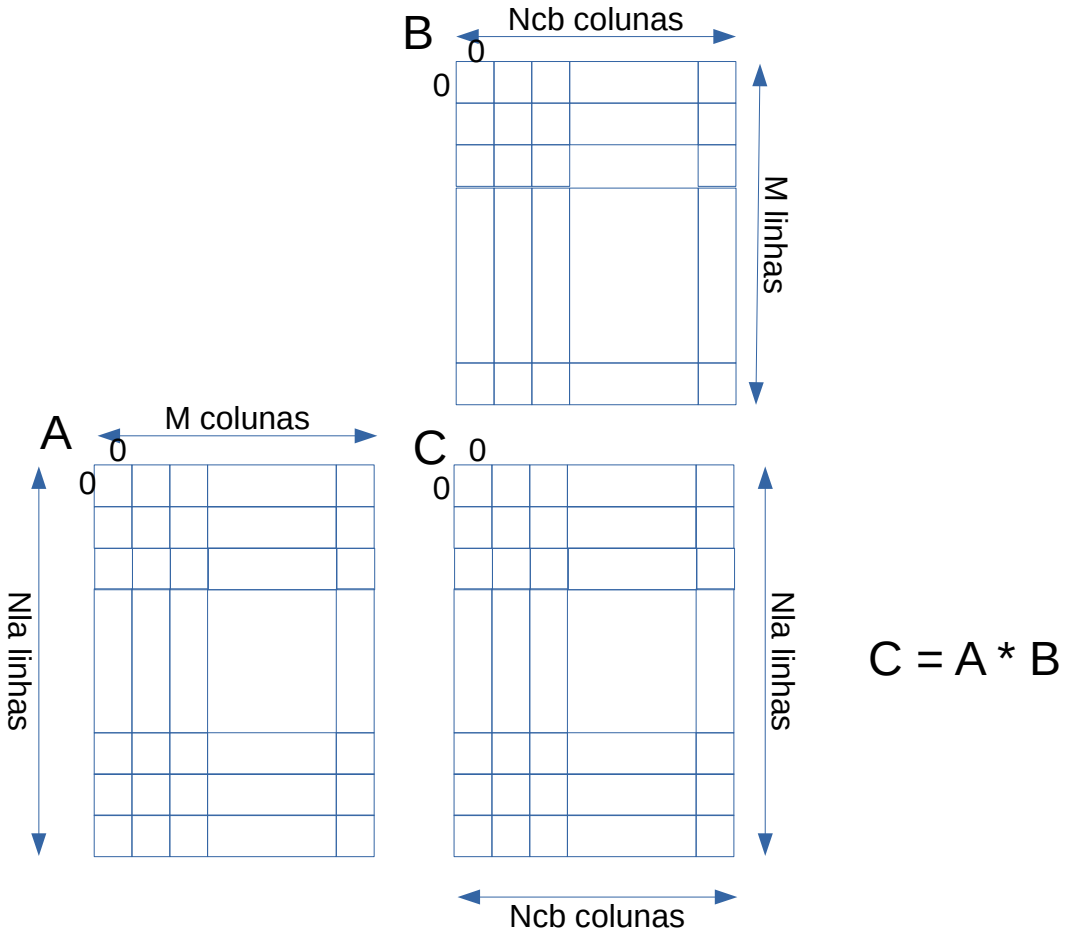
```
if (rank == 0) {
    srand(1);                          /* semente fixa: execuções reproduzíveis */
    gera_matriz(A, (size_t)Nla * M);       /* A [Nla x M] */
    gera_matriz(B, (size_t)M * Ncb);       /* B [M x Ncb] */
}
```

Dois pontos que valem uma linha no enunciado:

- **A ordem das chamadas importa.** Gerar A e depois B dá matrizes diferentes de gerar B e depois A, porque as duas usam a mesma sequência do `rand()`. Siga a ordem acima.
- **A geração deve ficar fora da medição de tempo.**

..... segue

- OBS: será na página seguinte temos a ilustraçao do prolema



Envio das matrizes e processamento nos nodos, **na versao A, note que a versao A, não tem otimizacao nem para uso de cache nem para otimizacao de comunicacao** (ou seja, a versao A usará comunicacao bloqueante apenas)

Exemplo para 3 processos MPI

Breve descriçao da versao A:

- Processo 0 preenche matrizes A e B
- Processo 0 faz MPI_Scatter de 3 faixas horizontais (de linhas) de A, cada processo recebe uma faixa
- Processo 0 faz MPI_Broadcast de B para os outros nodos
- Cada processo MPI multiplica sua faixa horizontal de A com suas colunas necessárias de B. Assim cada processo obtem sua faixa horizontal da multiplicacao em C
- Os processos fazem MPI_Gather da matriz C para destino no nodo 0
- Ao final o processo 0 tem as 3 matrizes completas, sendo C = A * B